



NOMBRE DEL ESTUDIANTE:

Francisco Xavier Gil Ginez

DOCENTE:

José Miguel Carrera Pacheco

MATERIA:

Desarrollo Web Profesional

CUATRIMESTRE ENERO-ABRIL

2026

JavaScript Asíncrono

La programación asíncrona en JavaScript es un mecanismo fundamental para gestionar operaciones que no pueden completarse de manera inmediata, como solicitudes a servidores, lectura de archivos o consultas a bases de datos. Debido a que JavaScript opera bajo un modelo de ejecución monohilo, no puede ejecutar múltiples tareas pesadas simultáneamente sin bloquear la interfaz. Para resolver esto, utiliza el Event Loop, que permite delegar tareas externas mientras el flujo principal continúa ejecutándose. Inicialmente, la asincronía se manejaba mediante callbacks, lo que generaba problemas de mantenimiento debido a la anidación excesiva. Posteriormente surgieron las Promises, que permiten representar el resultado futuro de una operación, facilitando el encadenamiento lógico de procesos. Finalmente, la introducción de `async/await` permitió estructurar estas operaciones de manera más legible, simulando un flujo secuencial sin perder la naturaleza no bloqueante. Este modelo es ampliamente utilizado en entornos como Node.js, donde la eficiencia depende directamente de la correcta gestión de procesos asíncronos.

APIs REST

Las APIs REST constituyen un estilo arquitectónico que permite la comunicación entre sistemas distribuidos mediante el protocolo HTTP. En este modelo, los datos se estructuran como recursos identificables mediante URLs, los cuales pueden manipularse utilizando métodos estándar como GET, POST, PUT, PATCH y DELETE. Una de sus principales características es que son sin estado, lo que implica que cada solicitud contiene toda la información necesaria para ser procesada, sin depender del contexto previo del servidor. Este enfoque facilita la escalabilidad y la interoperabilidad entre sistemas. En la práctica, las aplicaciones modernas utilizan APIs REST para autenticación, sincronización de información, almacenamiento remoto y ejecución de servicios externos. La adopción de este modelo fue formalizada por Roy Fielding en su tesis doctoral, estableciendo las bases de la comunicación web moderna.

Estados de Carga Accesibles

Cuando una aplicación realiza operaciones asíncronas, existe un intervalo de tiempo en el que el resultado aún no está disponible. Durante este periodo, el sistema entra en un estado de carga. Desde la perspectiva de experiencia de usuario, es fundamental comunicar que la operación está en progreso. Sin embargo, no basta con mostrar indicadores visuales como animaciones; el estado debe ser accesible para todos los usuarios, incluyendo aquellos que utilizan tecnologías asistivas. Esto implica que el sistema debe transmitir programáticamente que una acción se encuentra en ejecución. Las directrices promovidas por el World Wide Web Consortium establecen que los cambios dinámicos en la interfaz deben ser interpretables más allá del componente visual. Un estado de carga accesible reduce la incertidumbre del usuario, mejora la percepción de estabilidad del sistema y evita la interpretación errónea de fallos.

Manejo de Errores de Red

La dependencia de servicios externos introduce múltiples puntos de fallo dentro de una aplicación. Los errores de red pueden surgir por pérdida de conectividad, tiempos de espera, respuestas inválidas o problemas del servidor. El manejo adecuado de estos errores es crucial para garantizar la resiliencia del sistema. No se trata únicamente de capturarlos, sino de responder de forma que la aplicación continúe siendo funcional y comprensible. La programación asíncrona permite interceptar fallos mediante mecanismos como try/catch o la gestión de estados de Promises. Además, desde una perspectiva de diseño robusto, es recomendable implementar estrategias como reintentos automáticos, almacenamiento temporal de datos o mensajes informativos que orienten al usuario. Un manejo eficiente de errores evita bloqueos, mantiene la continuidad operativa y mejora la confianza en el sistema.

Referencias

Mozilla. (2024). *MDN Web Docs – Asynchronous JavaScript*.

Mozilla. (2024). *MDN Web Docs – Fetch API*.

Node.js. (2024). *Official Documentation*.

Google Developers. (2023). *Web Fundamentals: Network Resilience*.

Nielsen, J. (2012). *Usability Engineering*. Morgan Kaufmann.